

《软件测试技术与实践》课程

第四讲 软件生命周期相关测试（一）

中国科学院大学计算机科学与技术学院

2026.09.22

（仅用于教学，请勿网络传播）

1

本讲学习目的与内容

1 本讲学习目的与内容

2 A 从软件生命周期角度来看测试

- A.1 软件测试过程模型的发展
- A.2 实施软件测试

3 B 单元测试

- B.1 理解单元测试
- B.2 单元测试执行环境
- B.3 程序插桩技术
- B.4 单元测试工具的运用

■ B.5 本部分小结与思考

4 C 集成测试

- C.1 理解集成测试
- C.2 集成测试策略
- C.3 本部分小结与思考

5 D 案例：使用 Google Test 进行单元测试

- D.1 Google Test 测试框架
- D.2 理解 GTest 测试代码编写
- D.3 实例：搭建 GTest 测试环境及测试

6 课后思考及工程实践

例 1：消费账单折扣计算函数

消费账单折扣规则

- ▶ 当账单上的一次性消费额为负数或零时，返回负数表示消费数额无效；
- ▶ 当消费额在 800 元到 1800 元之间时 (不含 800 元，但包含 1800 元)，为 9 折优惠；
- ▶ 当消费额在 1800 元到 4800 元之间时 (含 4800 元)，为 8 折优惠；
- ▶ 当消费额在 4800 元以上时 (不含 4800 元)，一律为 7 折优惠。

思考

- ▶ 采用什么方法来对上述规则进行测试？

例 1：消费账单折扣计算函数

消费账单折扣规则

- ▶ 当账单上的一次性消费额为负数或零时，返回负数表示消费数额无效；
- ▶ 当消费额在 800 元到 1800 元之间时 (不含 800 元，但包含 1800 元)，为 9 折优惠；
- ▶ 当消费额在 1800 元到 4800 元之间时 (含 4800 元)，为 8 折优惠；
- ▶ 当消费额在 4800 元以上时 (不含 4800 元)，一律为 7 折优惠。

思考

- ▶ 采用什么方法来对上述规则进行测试？
- ▶ 使用等价类划分和边界值分析法来测试。

例 1：消费账单折扣计算函数

消费账单折扣规则

- ▶ 当账单上的一次性消费额为负数或零时，返回负数表示消费数额无效；
- ▶ 当消费额在 800 元到 1800 元之间时 (不含 800 元，但包含 1800 元)，为 9 折优惠；
- ▶ 当消费额在 1800 元到 4800 元之间时 (含 4800 元)，为 8 折优惠；
- ▶ 当消费额在 4800 元以上时 (不含 4800 元)，一律为 7 折优惠。

思考

- ▶ 采用什么方法来对上述规则进行测试？
- ▶ 使用等价类划分和边界值分析法来测试。
- ▶ 使用基本路径测试法来覆盖独立路径。

从软件测试角度来看，你会想到什么问题？

1、前面两讲中所学的测试方法，似乎主要针对函数（或过程）、或者代码规模较小的独立模块？

- ▶ 实际上，它不是直接针对函数（或过程）或者软件模块，而是用来做测试设计的“测试方法”，对应测试思路可以应用于各种场景或需求中。
- ▶ 单单使用这些常用的软件测试方法，不足以保障软件系统的质量目标，并且不知道什么时机做、如何做！

从软件测试角度来看，你会想到什么问题？

2、如果某一软件包含较大规模的函数或独立模块，对某一函数或模块需要设计数十甚至数百个测试用例，那么，对大规模的软件或安全攸关的软件，其测试用例数量会很大，如何开展验证？

- ▶ No silver bullet! 复杂的软件工程问题无法靠简单的方法来解决。
- ▶ 需要从软件工程角度来寻找实际上可行、可操作的做法。
- ▶ 软件测试与软件开发过程紧密协同，并且软件测试“应以小规模开始，逐步转向大规模”。

如何实现软件测试与开发紧密协同？

自然会想到：

- ▶ 需要从软件生命周期过程角度控制各阶段、各层次的软件质量。
- ▶ 在软件生命周期的不同阶段，关注的质量因素不同，因而，测试关注点和内容也不一样，所用测试方法也有不同。

因此，也有必要建立软件测试过程模型：

- ▶ 一个好的软件测试过程模型，应与软件开发过程匹配；
- ▶ 既能检验一小段代码是否正确，又能验证整个系统是否符合用户需求；
- ▶ 同时，在不同的阶段应用不同的测试技术，使测试尽可能多地发现软件错误，从而保证软件质量。

这就说明了**我们为什么需要从软件生命周期角度来理解软件测试。**

从软件生命周期角度来理解软件测试（“智能”的客服机器人）

某公司引入 AI 大模型，开发一套智能客服机器人：

- ▶ 阿码（开发）：接入了最新大模型 API，做了精美的对话界面，支持多轮对话、意图识别、情感分析。技术架构先进，代码质量高。
- ▶ 小测（测试）：测试了机器人的对话流畅度、响应速度，各项指标都达标。

上线后，用户投诉不断：

- ▶ ???

从软件生命周期角度来理解软件测试（“智能”的客服机器人）

某公司引入 AI 大模型，开发一套智能客服机器人：

- ▶ 阿码（开发）：接入了最新大模型 API，做了精美的对话界面，支持多轮对话、意图识别、情感分析。技术架构先进，代码质量高。
- ▶ 小测（测试）：测试了机器人的对话流畅度、响应速度，各项指标都达标。

上线后，用户投诉不断：

- ▶ ???
- ▶ 用户问“怎么退款”，机器人回答“根据《消费者权益保护法》第 XX 条……”

从软件生命周期角度来理解软件测试（“智能”的客服机器人）

某公司引入 AI 大模型，开发一套智能客服机器人：

- ▶ 阿码（开发）：接入了最新大模型 API，做了精美的对话界面，支持多轮对话、意图识别、情感分析。技术架构先进，代码质量高。
- ▶ 小测（测试）：测试了机器人的对话流畅度、响应速度，各项指标都达标。

上线后，用户投诉不断：

- ▶ ???
- ▶ 用户问“怎么退款”，机器人回答“根据《消费者权益保护法》第 XX 条……”
- ▶ 用户说“我生气了”，机器人回答“检测到您的情绪为负面，建议深呼吸”

从软件生命周期角度来理解软件测试（“智能”的客服机器人）

某公司引入 AI 大模型，开发一套智能客服机器人：

- ▶ 阿码（开发）：接入了最新大模型 API，做了精美的对话界面，支持多轮对话、意图识别、情感分析。技术架构先进，代码质量高。
- ▶ 小测（测试）：测试了机器人的对话流畅度、响应速度，各项指标都达标。

上线后，用户投诉不断：

- ▶ ???
- ▶ 用户问“怎么退款”，机器人回答“根据《消费者权益保护法》第 XX 条……”
- ▶ 用户说“我生气了”，机器人回答“检测到您的情绪为负面，建议深呼吸”
- ▶ 用户问“你们的投诉电话是多少”，机器人编造了一个不存在的号码！

从软件生命周期角度来理解软件测试（“智能”的客服机器人）

复盘发现：

- ▶ 阿码不懂业务——他没有和业务部门对齐客服的标准话术和业务流程，只是接入了通用大模型，没有做领域微调；
- ▶ 小测不懂业务——她的测试用例只覆盖了“技术指标”（准确率、响应时间），没有覆盖“业务场景”（退款流程、投诉处理、转人工规则）；
- ▶ 运哥（运维）不懂业务——他没有配置人工接管机制，当机器人无法回答时，用户只能对着屏幕干着急。

启示：

- ▶ 所有学科都离不开业务，学习软件测试也就是要学习测试业务。
- ▶ 不管什么样的测试方法、技术、工具，都是面对同一个对象：一个软件的全生命周期。
- ▶ 软件测试无法回避单元测试、集成测试、系统测试等。

基础知识学习

学习目的

- ▶ 理解单元测试和相关测试技术
- ▶ 理解集成测试和各种集成测试策略

学习内容

- ▶ A 从软件生命周期角度来看测试
- ▶ B 单元测试
- ▶ C 集成测试

测试工程实践

学习目的

- ▶ 练习单元测试环境搭建以及测试代码编写

学习内容

- ▶ D 案例：使用 Google Test 进行单元测试

2

A 从软件生命周期角度来看测试

1 本讲学习目的与内容

2 A 从软件生命周期角度来看测试

- A.1 软件测试过程模型的发展
- A.2 实施软件测试

3 B 单元测试

- B.1 理解单元测试
- B.2 单元测试执行环境
- B.3 程序插桩技术
- B.4 单元测试工具的运用

■ B.5 本部分小结与思考

4 C 集成测试

- C.1 理解集成测试
- C.2 集成测试策略
- C.3 本部分小结与思考

5 D 案例：使用 Google Test 进行单元测试

- D.1 Google Test 测试框架
- D.2 理解 GTest 测试代码编写
- D.3 实例：搭建 GTest 测试环境及测试

6 课后思考及工程实践

1 本讲学习目的与内容

2 A 从软件生命周期角度来看测试

■ A.1 软件测试过程模型的发展

■ A.2 实施软件测试

3 B 单元测试

■ B.1 理解单元测试

■ B.2 单元测试执行环境

■ B.3 程序插桩技术

■ B.4 单元测试工具的运用

■ B.5 本部分小结与思考

4 C 集成测试

■ C.1 理解集成测试

■ C.2 集成测试策略

■ C.3 本部分小结与思考

5 D 案例：使用 Google Test 进行单元测试

■ D.1 Google Test 测试框架

■ D.2 理解 GTest 测试代码编写

■ D.3 实例：搭建 GTest 测试环境及测试

6 课后思考及工程实践

软件测试过程模型概述

在软件开发实践过程中，相继出现了各种软件开发过程模型：

- ▶ 边做边改模型
- ▶ 瀑布模型
- ▶ 快速原型模型
- ▶ 螺旋模型
- ▶ 增量模型
- ▶ 敏捷开发模型，……

开发过程模型没有给予测试足够的重视和诠释。

因此，提出了各种软件测试过程模型，对开发和测试进行了不同角度的融合：

- ▶ V 模型，W 模型，H 模型，X 模型
- ▶ 前置模型
- ▶ 测试驱动开发，等等

这些模型：

- ▶ 遵循软件工程原理和管理学原理。
- ▶ 根据软件生产实践，总结了测试与开发之间的关系，使测试过程与开发产生交互，是测试过程管理的重要参考依据。

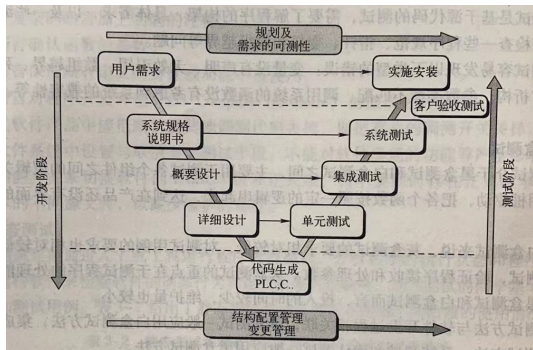
V 模型

对应：传统的软件生命周期模型

- ▶ 开发与测试相对分离。

历史局限性：

- ▶ 仅把测试过程作为在需求分析、概要设计、详细设计及编码之后的一个阶段，主要针对程序来发现错误。
- ▶ 忽视了测试活动对需求分析、系统设计等阶段的验证和确认功能。



W 模型

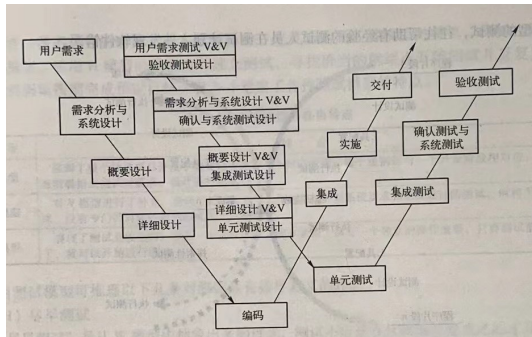
对应：传统的软件生命周期模型

增强：

- ▶ 增加了软件各开发阶段中应同步进行的验证和确认 (V&V) 活动
- ▶ 两个 V 字模型分别代表测试与开发过程，明确测试与开发的并行关系
- ▶ 不仅仅是程序，需求、设计等同样也需要测试

历史局限性

- ▶ 需求、设计、编码等活动被视为串行，测试和开发活动保持线性的前后关系
- ▶ 无法支持迭代开发



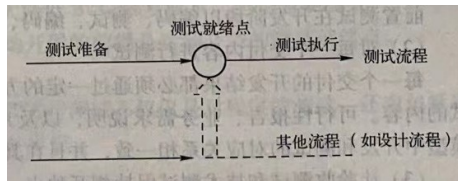
H 模型

观察实际的软件项目：

- ▶ 需求、设计、编码等在大部分时间内交叉
- ▶ 测试也不存在严格的次序关系

因此认为：

- ▶ 软件测试作为一个独立的流程贯穿于软件整个生命周期，与其他流程并发地进行
- ▶ 不同的测试活动可以按照某个次序先后进行，也可能是反复的，只要某个测试达到准备就绪点，测试执行活动就可以开展
- ▶ 软件测试要尽早准备，尽早执行



H 模型将测试活动完全独立出来：

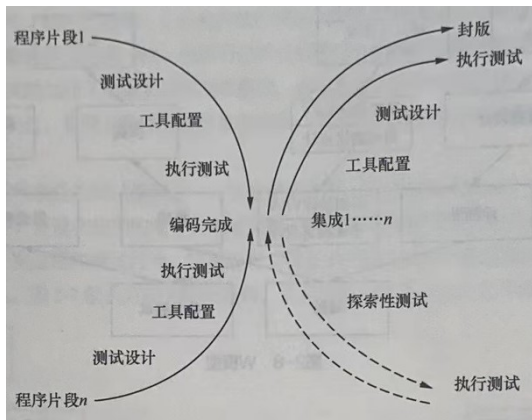
- ▶ 测试准备与测试执行分离，有利于资源调配，降低成本，提高效率
- ▶ 充分体现测试过程（不是技术）的复杂性

X 模型

X 模型：对应增量模型

特点：体现出测试设计、测试回溯的过程

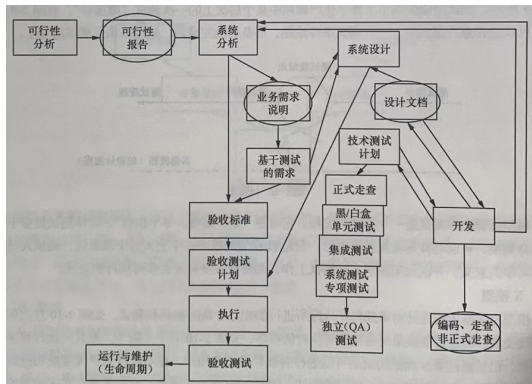
- ▶ 针对单独程序片段进行编码和测试；
- ▶ 通过频繁的交互、集成，最终生成为可执行的程序；
- ▶ 已通过集成测试的成品进行封版并提交给用户，也可以作为更大规模和范围内集成的一部分（多根并行的曲线表示）；
- ▶ 定义了探索性测试作为测试补充（不进行事先计划）。



前置模型

前置测试：测试和开发紧密结合

- ▶ **开发和测试相结合**：将开发和测试的生命周期整合，标识了关键行为
- ▶ **对每一个交付内容都必须进行测试**：包括源程序代码、可行性报告、业务需求说明，以及系统设计文档等
- ▶ **验收测试和技术测试保持相互独立**：保证设计及程序编码能够符合最终用户的需求
- ▶ **反复交替地开发和测试**：软件开发中存在很多变更，开发和测试需要一起反复交替地执行



(图中椭圆表示除代码外的其它测试对象)

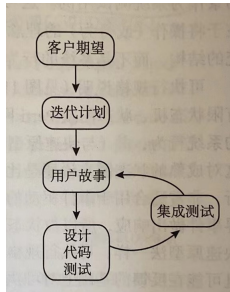
测试驱动开发 (Test-driven development, TDD)

对应：敏捷开发模型

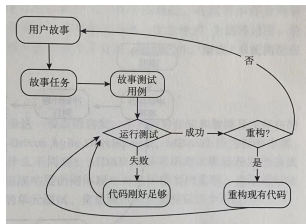
- ▶ 以客户为导向，自底向上开发
- ▶ 尽早发布功能完整的组件
- ▶ 灵活应对需求变更

测试驱动开发

- ▶ 用户故事可以被分解为多项故事任务
- ▶ 开发人员编写针对任务的故事测试用例（相当于规格说明）
- ▶ 进行测试；如果未通过则添加代码，然后再测试，一直重复
- ▶ 直到一个新的用户故事被实现



敏捷开发社区认为，一个好的设计来自于反复的重构。



软件测试过程模型总结及指导原则

软件测试过程模型都有局限性，与开发过程模型一样，测试过程模型也不断发展。在实际工作中应灵活选用各种模型的优点。可参考的指导原则：

1、尽早测试

- ▶ 测试人员早期参与软件项目，及时开展测试的准备工作（测试设计）；
- ▶ 尽早地开展测试执行工作，一旦代码模块完成就应该及时开展单元测试，一旦代码模块被集成为相对独立的子系统，便可以开展集成测试等。

2、全面测试

- ▶ 软件是程序、数据和文档的集合，应对软件的所有产品进行全面的测试；
- ▶ 软件开发人员、测试人员和用户都参与到测试工作中。

软件测试过程模型总结及指导原则

3、全过程测试

- ▶ 测试人员要充分关注开发过程，对开发过程的各种变化及时作出响应；
- ▶ 测试人员要对测试进行全程的跟踪，建立完善的度量与分析机制，对过程进行度量，及时了解过程信息，调整测试策略。

4、独立的、迭代的测试

- ▶ 在软件开发过程中的螺旋、迭代等诸多模型中，需求、设计、编码工作必然是重叠并反复进行的，这时的测试工作也是迭代和反复的；
- ▶ 必须将测试从开发中抽象出来进行管理，否则测试管理将陷入困境。

1 本讲学习目的与内容

2 A 从软件生命周期角度来看测试

■ A.1 软件测试过程模型的发展

■ A.2 实施软件测试

3 B 单元测试

■ B.1 理解单元测试

■ B.2 单元测试执行环境

■ B.3 程序插桩技术

■ B.4 单元测试工具的运用

■ B.5 本部分小结与思考

4 C 集成测试

■ C.1 理解集成测试

■ C.2 集成测试策略

■ C.3 本部分小结与思考

5 D 案例：使用 Google Test 进行单元测试

■ D.1 Google Test 测试框架

■ D.2 理解 GTest 测试代码编写

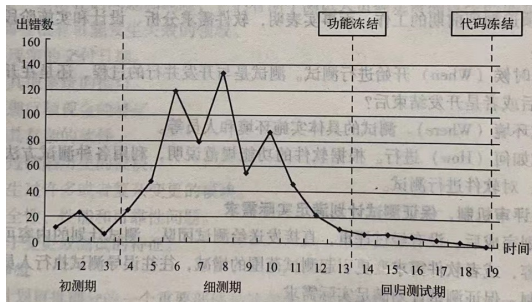
■ D.3 实例：搭建 GTest 测试环境及测试

6 课后思考及工程实践

软件测试的实施阶段

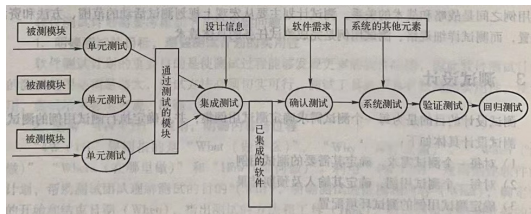
大致可划分为三个实施阶段：

- ▶ 初测期：测试软件的主要功能模块和关键的执行路径，排除主要障碍。（可看作**冒烟测试**）
- ▶ 细测期：依据测试计划、测试大纲和测试用例，逐一测试软件的功能、性能、用户界面、兼容性等多个方面。
- ▶ 回归测试期：复查已知错误的纠正情况；在未引发任何新的错误时，结束测试。



在软件生命周期的不同阶段，有不同测试重点，实施不同类型的测试

- ▶ **验证、回归测试**: 重新执行测试项的某个子集, 保证没有引入新的错误
- ▶ **验收测试**: 由用户对软件进行产品质量检测



3

B 单元测试

1 本讲学习目的与内容

2 A 从软件生命周期角度来看测试

- A.1 软件测试过程模型的发展
- A.2 实施软件测试

3 B 单元测试

- B.1 理解单元测试
- B.2 单元测试执行环境
- B.3 程序插桩技术
- B.4 单元测试工具的运用

■ B.5 本部分小结与思考

4 C 集成测试

- C.1 理解集成测试
- C.2 集成测试策略
- C.3 本部分小结与思考

5 D 案例：使用 Google Test 进行单元测试

- D.1 Google Test 测试框架
- D.2 理解 GTest 测试代码编写
- D.3 实例：搭建 GTest 测试环境及测试

6 课后思考及工程实践

1 本讲学习目的与内容

2 A 从软件生命周期角度来看测试

- A.1 软件测试过程模型的发展
- A.2 实施软件测试

3 B 单元测试

- B.1 理解单元测试
- B.2 单元测试执行环境
- B.3 程序插桩技术
- B.4 单元测试工具的运用

■ B.5 本部分小结与思考

4 C 集成测试

- C.1 理解集成测试
- C.2 集成测试策略
- C.3 本部分小结与思考

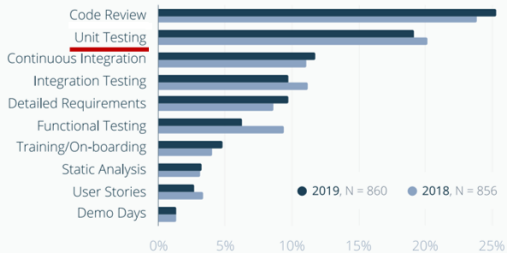
5 D 案例：使用 Google Test 进行单元测试

- D.1 Google Test 测试框架
- D.2 理解 GTest 测试代码编写
- D.3 实例：搭建 GTest 测试环境及测试

6 课后思考及工程实践

软件测试 “应以小规模开始，逐步转向大规模”

What is the #1 thing a company can do to improve code quality?
fig.4



$$0.8^{30} = 0.00123$$

$$0.99^{30} = 0.740$$

软件测试“应以小规模开始，逐步转向大规模”

逻辑上，与传统制造业类似：汽车工厂——“先检螺丝，再装整车”

- ▶ 在汽车制造流水线上，不会等一辆车完全组装好才去检查质量。而是在每一个零件下线时，就对其进行独立检验：这颗螺丝的扭矩是否达标？这个齿轮的齿数是否正确？这块电路板的焊点是否牢固？
- ▶ 如果等到整车下线后才发现某个齿轮有问题，就需要拆解整辆车才能定位和修复，成本是检测零件阶段的几十倍甚至上百倍。

单元测试：

- ▶ 就是**软件领域的“零件质检”**；
- ▶ 在每个函数、方法编写完成后立即验证其正确性，确保每一块“零件”都是合格的，再进入下一阶段的组装。

软件单元

软件单元 (Software Unit): 一个应用程序中的最小可测部分

- ▶ 对于面向过程的开发语言来说, 单元常指单独的程序、函数、过程、网页、菜单、……
- ▶ 对于面向对象的开发语言来说, 单元常指一个类 (基类、抽象类、子类)
- ▶ 图形化软件中, 单元常指一个窗口或一个菜单、……
- ▶ …… (具体问题具体分析)

什么是单元测试？

单元测试 (Unit Testing)

- ▶ 对最低层的、可测试的软件单元进行验证，保证软件单元符合详细设计说明要求。

验证目标：

- ▶ 设计输入主要有软件单元说明、详细设计说明和代码等。
- ▶ 在某些情况下也可称为**模块测试** (Module Testing) 或**组件测试** (Component Testing)。

实施者：

- ▶ 测试人员？

1. **单元本身：被正确实现**（主要指编码），包括功能、性能、安全性等，但一般主要关注单元功能测试。
2. **输入输出正常：被正确传递和得到保护（容错）**
3. **数据完整：内部数据能够保持其完整性**，包括变量的正确定义与引用、内存及时释放、全局变量的正确处理和影响最低。

什么是单元测试？

单元测试 (Unit Testing)

- ▶ 对最低层的、可测试的软件单元进行验证，保证软件单元符合详细设计说明要求。

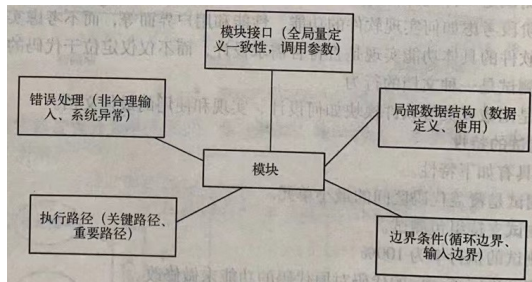
验证目标：

- ▶ 设计输入主要有软件单元说明、详细设计说明和代码等。
 - ▶ 在某些情况下也可称为**模块测试** (Module Testing) 或**组件测试** (Component Testing)。
- ### 实施者：
- ▶ 测试人员？
 - ▶ 软件开发人员！
1. **单元本身：被正确实现**（主要指编码），包括功能、性能、安全性等，但一般主要关注单元功能测试。
 2. **输入输出正常：被正确传递和得到保护（容错）**
 3. **数据完整：内部数据能够保持其完整性**，包括变量的正确定义与引用、内存及时释放、全局变量的正确处理和影响最低。

有哪些具体的验证内容？

围绕验证目标，主要的验证内容：

1. 模块接口
2. 局部数据结构
3. 边界条件
4. 执行路径
5. 错误处理



验证内容 1：软件单元的接口参数

目标：

- ▶ 进出软件单元（模块/组件）的数据流正确

为什么做接口参数验证？

- ▶ 只有在数据能正确流入、流出软件单元的前提下，其他测试才有意义。
- ▶ 检查软件单元的接口参数，是单元测试的基础。

关注点：

- ▶ 接口名称、参数个数、类型、匹配顺序
- ▶ 输出或返回值及其类型是否正确

验证内容 1：软件单元的接口参数（典型因素）

典型的因素，例如：

1. 输入的实际参数与形式参数个数是否相同
2. 输入的实际参数与形式参数属性是否匹配
3. 调用其他组件时所给实际参数个数是否与被调组件的形参个数相同。
4. 调用其他组件时所给实际参数属性是否与被调组件的形参属性匹配。
5. 调用预定义函数时所用参数的个数、属性和次序是否正确。
6. 是否存在与当前入口点无关的参数引用。
7. 是否修改了只读型参数。
8. 各组件对全程变量的定义是否一致。

如果组件包括输入输出，还应该考虑以下因素：

1. 文件属性是否正确。
2. OPEN/CLOSE 语句是否正确。
3. 格式说明与输入输出语句是否匹配。
4. 缓冲区大小与记录长度是否匹配。
5. 文件使用前是否已经打开。
6. 是否处理了文件尾。
7. 是否处理了输入/输出错误。
8. 输出信息中是否有文字性错误。

验证内容 2：局部数据结构

目标

- ▶ 应保证临时存储在组件内的数据完整正确。

要求：

- ▶ 数据在模块执行过程中维持完整性和正确性

关注点：

- ▶ 数据结构定义和使用过程的正确性
- ▶ 局部数据结构对全局数据结构的影响

典型检测因素或错误：

1. 不合适或不相容的类型说明。
2. 变量无初值。
3. 变量初始化有错或缺省值有错。
4. 不正确的变量名（拼错或不正确的截断）。
5. 出现上溢、下溢和地址异常。

验证内容 3：执行路径

目标：

- ▶ 为了发现因错误计算、不正确的条件和不当的控制流造成的错误，应对每一条独立执行路径进行测试。

要求：

- ▶ 通常要保证模块中的每条独立路径（基本路径）都被覆盖，使得所有语句都被执行过一次

关注点：

- ▶ 对路径的选择性测试（基本路径测试 + 针对循环的测试）

注意检查以下错误：

1. 误解或用错了逻辑运算符或优先级。
2. 错误的变量初值。
3. 数据精度不够（如因计算机表示的局限性，期望理论上相等而实际上不相等的错误）。
4. 错误的表达式符号。
5. 不可能出现的循环终止条件（死循环）。
6. 错误地修改了循环变量。

验证内容 4：边界条件

目标：

- ▶ 保证模块在条件边界上能够正确执行

注意检查以下类别的边界情况：

软件经常在边界上失效，
出现的错误防不胜防。

关注点：

- ▶ 数据结构中的边界
(例如：数组 $a[n]$)
- ▶ 控制流中的边界
(例如：循环次数、
判断条件)

1. 一致（实际数值是否和预期的一致）
2. 顺序（实际数值是否像预期的那样有序或者无序）
3. 区间（实际值是否位于合理的最小值和最大值之内）
4. 依赖（代码是否引用了一些不在代码本身控制范围之内的外部资源）
5. 存在（实际数值是否非 Null、非 0、在一个集合中，等等）
6. 相对或者绝对的时间（所有动作发生是否有序？是否正确
的时刻？是否及时？）

验证内容 5：错误处理

出错处理同样需要经过测试

- ▶ 好的软件设计应能预见各种出错条件，并预设各种出错处理；
- ▶ 当错误发生时，错误处理路径被建立，以重定向或终止处理。

目标：

- ▶ 验证错误处理的正确性，保证软件的健壮性

关注点：

- ▶ 异常条件的正确性
- ▶ 异常的可发生性
- ▶ 异常处理的逻辑正确性

应着重检查以下问题：

1. 输出的出错信息是否难以理解。
2. 记录的错误与实际遇到的错误是否相符。
3. 异常处理不当。
4. 错误陈述中未能提供足够的出错定位信息。

单元测试的主要测试方法及测试充分性

在单元测试中，可以使用前两讲的常用测试方法：

- ▶ 基于输入域的测试方法：等价类划分法、边界值分析法、决策表、因果图法、Pairwise、正交试验法
- ▶ 基于代码的测试方法：控制流覆盖、路径覆盖方法
- ▶ 其他方法

针对不同的测试目的，采用不同方法：

- ▶ 单元体现了预期的功能（黑盒、白盒）
- ▶ 单元的运行能覆盖预定的各种逻辑（白盒）
- ▶ 单元的内部数据能够保持完整性（白盒）
- ▶ 可接受正确数据和处理非法数据（黑盒）
- ▶ 在数据边界条件上能正确工作（黑盒）
- ▶ 单元的算法合理，性能良好（黑盒）
- ▶ 扫描代码未发现任何安全性问题（白盒）
- ▶

单元测试的主要测试方法及测试充分性

如何判断单元测试的充分性？

- ▶ 我们说测试是否充分，要依据某一测试充分性准则来度量，但是，如果没有考虑这一测试充分性准则是什么，就很难回答测试是否“足够”。
- ▶ 因此，需要在项目开始时就根据业务需求来明确基于测试的需求：其中规定**单元测试的覆盖要求**；或者在整个测试项目的验收标准中明确出单元覆盖要求。

那么，在项目实施过程中，如何确定出测试“足够”充分的判定条件？

- ▶ 综合考虑：用户需求，要遵循的行业标准或国家标准。

例 2: 某行业关于单元测试的技术要求

为保证软件单元测试充分和有效,在进行软件单元测试时,应满足以下技术要求:

(1) 对软件设计文档规定的软件单元的功能、性能、接口等要求应逐

项进行测试,不能有遗漏。

(2) 每个软件特性应至少被一个正常测试用例和一个被认可的异常测试用例覆盖。

(3) 测试用例(见第5章)的输入应至少包括有效等价类值、无效等价类值和边界数据值。

(4) 在对软件单元进行动态测试之前,应对软件单元的源代码进行静态测试。

(5) 应进行语句覆盖测试。对于安全级别较高的软件(例如 A、B 级软件),语句覆盖率应达到 100%。还应对没有覆盖的语句进行分析,给出理由。

(6) 应进行分支覆盖测试。对于安全级别较高的软件(例如 A、B 级软件),分支覆盖率应达到 100%。还应对没有覆盖的分支进行分析,给出理由。

(7) 对用高级语言编制的安全级别较高的软件(例如 A、B 级软件)需进行修正的条件判定覆盖测试。

(8) 对用高级语言编制的安全级别较高的(例如 A、B 级)嵌入式软件,需对目标码进行语句覆盖和分支覆盖测试,覆盖率均应达到 100%。

(9) 对输出数据及其格式进行测试。

(10) 对有特殊要求的软件单元,应对其占用空间、运行时间和计算精度等内容进行测试。

单元测试停止的条件

单元测试停止：

- ▶ 意味着测试工作到达了集成测试的“测试就绪点”。

示例：针对某行业项目的单元测试停止条件

1. 单元测试用例设计已经通过评审。
2. 按照单元测试计划完成了所有规定单元的测试。
3. 达到了测试计划中关于单元测试所规定的覆盖率的要求。
4. 软件单元功能与设计相一致。
5. 在单元测试中发现的错误已经得到修改，各级缺陷修复率达到标准。

单元测试的价值

- ▶ 编程与编译运行结束后，每 1000 行代码中大约残留有 2-6 个 Bug
- ▶ 寻找与修改程序错误的代价占总体开发投资的 30% -60%
- ▶ 尽早发现错误，成本越低：发现问题比较容易；修正问题更容易
- ▶ 单元质量是系统质量的基石，单元测试可以比系统测试测得更彻底

1 本讲学习目的与内容

2 A 从软件生命周期角度来看测试

- A.1 软件测试过程模型的发展
- A.2 实施软件测试

3 B 单元测试

- B.1 理解单元测试
- B.2 单元测试执行环境
- B.3 程序插桩技术
- B.4 单元测试工具的运用

■ B.5 本部分小结与思考

4 C 集成测试

- C.1 理解集成测试
- C.2 集成测试策略
- C.3 本部分小结与思考

5 D 案例：使用 Google Test 进行单元测试

- D.1 Google Test 测试框架
- D.2 理解 GTest 测试代码编写
- D.3 实例：搭建 GTest 测试环境及测试

6 课后思考及工程实践

问题的提出

单元测试在编码阶段进行

- ▶ 在源程序代码编写完成、确认没有语法错误之后即可开始单元测试。

问题：

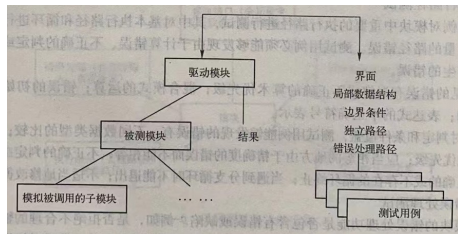
- ▶ 绝大多数情况下，单个软件单元是不能正常工作的，而是要和其他单元一起才能正常工作。然而，由于开发进度的原因，其他单元不一定已经开发完成，**在其它单元不存在的情况下，如何运行单个模块？**

这就需要我们能够构造某种测试执行环境来验证软件单元这个“半成品”，同时也需要能够达到单元测试的目的。

单元测试的执行环境

被测模块、驱动模块及桩模块共同构成了一个测试执行环境

- ▶ 由于软件模块并不是一个独立的程序，要考虑与其有关的外界联系，因此使用**辅助模块**去模拟与被测模块相关的其他模块。
- ▶ **驱动 (Driver) 模块/程序**：用于模拟被测模块的上一级模块，相当于被测模块的主程序，用于接收测试数据，并把这些数据传送给被测模块，启动被测模块，最后输出实测结果。
- ▶ **桩 (Stub) 模块/程序**：用于模拟被测模块工作过程中所调用的模块。桩模块一般只进行很少的数据处理，不需要把子模块的所有功能都带进来，但不允许什么事情也不做。



- ▶ **测试数据通过驱动模块传入被测模块。**

驱动模块与桩模块的代码结构

驱动模块

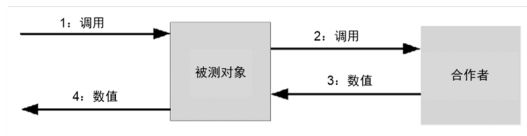
1. 数据说明;
2. 初始化;
3. 输入测试数据;
4. 调用被测模块;
5. 输出测试结果;
6. 停止

桩模块

1. 数据说明;
2. 初始化;
3. 输出提示信息; (进入了哪个桩模块)
4. 验证调用参数;
5. 打印验证结果;
6. 将模拟结果送回被测模块;
7. 返回

例 3：驱动程序与桩程序

```
1 //Program 1
2 #include <iostream.h>
3 void get_input(x& cost, int& y);
4 int main() ← Driver
5 {
6     double a;
7     int b;
8     char ans;
9     do
10     {
11         get_input(a, b); ← Function under test
12         cout.setf(ios::fixed);
13         cout.setf(ios::showpoint);
14         cout.precision(2);
15         cout << "a is " << a << endl;
16         cout << "b is " << b << endl;
17
18         cout << "Test again?"
19             << " (Type y for yes or n for no): ";
20         cin >> ans;
21         cout << endl;
22     } while (ans == 'y' || ans == 'Y');
23     return 0;
24 }
25 //Program 2
26 void function_under_test(int& x, int& y) {
27     ...
28     p = price(x);
29     ...
30 }
31 double price(int x) {return 10.00;} ← Stub
```



1 本讲学习目的与内容

2 A 从软件生命周期角度来看测试

- A.1 软件测试过程模型的发展
- A.2 实施软件测试

3 B 单元测试

- B.1 理解单元测试
- B.2 单元测试执行环境
- B.3 程序插桩技术
- B.4 单元测试工具的运用

■ B.5 本部分小结与思考

4 C 集成测试

- C.1 理解集成测试
- C.2 集成测试策略
- C.3 本部分小结与思考

5 D 案例：使用 Google Test 进行单元测试

- D.1 Google Test 测试框架
- D.2 理解 GTest 测试代码编写
- D.3 实例：搭建 GTest 测试环境及测试

6 课后思考及工程实践

问题的提出

技术需求：

- ▶ 在单元测试过程中，往往需要知道某些信息，如语句覆盖、路径覆盖以及变量的定义、使用情况等，以便给人工分析代码质量提供程序的状态信息及执行数据。
- ▶ 有些情况下需要知道测试用例执行过程中的代码覆盖率情况，以便为判断测试充分性提供数据依据。

早期的应对方法：

- ▶ 跟踪被测程序的执行过程、通过人工收集信息，这往往耗时耗力，当代码规模较大时基本不可能全面做到。

需要一种方法或技术，能够高效快速地获得程序执行过程中的相关信息。因而，研究人员提出了**程序插桩技术**。

程序插桩技术

程序插桩技术：

- ▶ 最早是由 J. C. Huang 教授提出的。
- ▶ 在保证被测程序原有逻辑完整性的基础上在程序中插入一些“探针”（即：桩），通过探针的执行并抛出程序运行的特征数据，通过对这些数据的分析，可以获得程序的控制流和数据流信息，进而得到逻辑覆盖等动态信息，从而实现测试目的的方法。

Huang, J. C. [Program Instrumentation and Software Testing](#)[J]. Computer, 1978, 11(4):25-32.



如何在测试中插桩？

基本操作思路：

- ▶ 在程序特定部位附加一些操作或功能，用来检验或获得程序运行结果以及执行特性，用以支持软件测试过程。

作用：

- ▶ 生成特定状态，检验状态的可达性
- ▶ 显示或读取内部数据或私有数据
- ▶ 监测不变数据
- ▶ 监测前提条件
- ▶ 人为触发事件
- ▶ 监测事件时间

如何在测试中插桩？

问题 1：探测什么信息？

回答：由具体需求而定

- ▶ 覆盖率？
- ▶ 中间结果？
- ▶ 测试数据生成？
- ▶ 统计信息？
- ▶

如何在测试中插桩？

问题 2：插桩位置？

回答：由具体需求而定

原则：

- ▶ 探针的植入要做到紧凑，才能保证在做到收集的信息全面而无冗余，减少代码的膨胀率

探针的常见位置：

- ▶ 程序的第一条语句
- ▶ 程序的结束语句
- ▶ 循环语句的开始
- ▶ 循环语句的结束
- ▶ 分支语句的开始
- ▶ 分支语句的结束
- ▶ 下一个入口语句之前的语句

通常是程序或基本块的开始和结束位置。

如何在测试中插桩？

问题 3：探测点数目？

回答：个数越少越好

- ▶ 插桩会影响原有程序执行的效率
- ▶ 很多测试工具能根据需求进行自动插桩

常用插桩策略：

- ▶ 语句覆盖插桩（基本块插桩）：在基本块的入口和出口处，分别植入相应的探针，以确定程序执行时该基本块是否被覆盖。
- ▶ 分支覆盖插桩：分支由分支点确定。对于每个分支，在其开始处植入一个相应的探针，以确定程序执行时该分支是否被覆盖。
- ▶ 条件覆盖插桩：if, switch, while, do-while, for 几种语法结构都支持条件判定，在每个条件表达式的布尔表达式处植入探针，进行变量跟踪取值，以确定其被覆盖情况。

例 4：统计程序流图中不同边的执行次数

被测函数

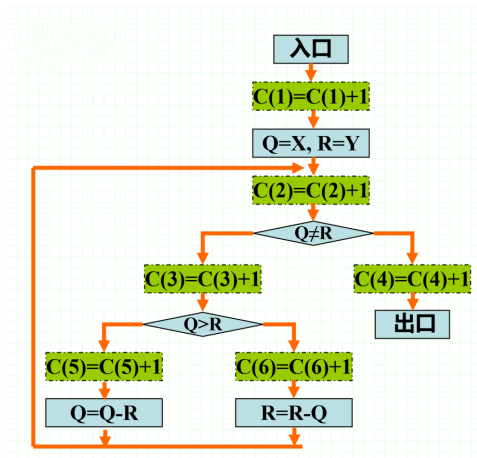
- ▶ 求整数 X 和 Y 的最大公约数函数

目标：

- ▶ 假设我们使用某一逻辑覆盖准则（如语句、判定或条件覆盖等）或路径覆盖方法来设计测试用例，希望分析测试执行的覆盖率以及测试用例有效性。

方法：

- ▶ 可以在程序流图中不同边的位置记录其遍历次数。



例 5：断言检测

断言：在程序特定部位插入用以证实程序某些特性的语句

```
1
2 void *memcpy(void *pvTo, const void *pvFrom, size_t size)
3 {
4     assert((pvTo != NULL) && (pvFrom != NULL));
5     byte *pbTo = (byte *) pvTo;
6     byte *pbFrom = (byte *) pvFrom;
7     while( size-- > 0) *pbTo ++ = *pbFrom ++ ;
8     return pvTo;
9 }
10
```

1 本讲学习目的与内容

2 A 从软件生命周期角度来看测试

- A.1 软件测试过程模型的发展
- A.2 实施软件测试

3 B 单元测试

- B.1 理解单元测试
- B.2 单元测试执行环境
- B.3 程序插桩技术
- B.4 单元测试工具的运用

■ B.5 本部分小结与思考

4 C 集成测试

- C.1 理解集成测试
- C.2 集成测试策略
- C.3 本部分小结与思考

5 D 案例：使用 Google Test 进行单元测试

- D.1 Google Test 测试框架
- D.2 理解 GTest 测试代码编写
- D.3 实例：搭建 GTest 测试环境及测试

6 课后思考及工程实践

运用工具的必要性

问题：

- ▶ 软件中的独立可测单元数量随软件规模增长，对其进行“充分”测试可能需要大量的测试用例，使用人工方式代价巨大。
- ▶ 如何根据软件开发实际情况，来合理控制软件测试投入？

思考：

- ▶ 开发或运用自动化测试工具是必然的选择。

最常用的单元测试工具

C++、Java 和 Python 语言的单元测试中，受欢迎的测试工具包括单元测试框架、单元测试用例自动生成工具以及代码覆盖率统计工具等。

- ▶ Google Test
- ▶ JUnit、TestNG
- ▶ pytest
- ▶ unittest
- ▶ Coverage.py
- ▶ EvoSuite
- ▶ Diffblue Cover
- ▶ JMockit
- ▶ JaCoCo
- ▶ gcov、lcov、gcovr

建议：结合自己熟悉的编程语言体系来练习

- ▶ C++：Google Test / gcov、lcov、gcovr
- ▶ JAVA：JUnit / JMockit / JaCoCo

1 本讲学习目的与内容

2 A 从软件生命周期角度来看测试

- A.1 软件测试过程模型的发展
- A.2 实施软件测试

3 B 单元测试

- B.1 理解单元测试
- B.2 单元测试执行环境
- B.3 程序插桩技术
- B.4 单元测试工具的运用

■ B.5 本部分小结与思考

4 C 集成测试

- C.1 理解集成测试
- C.2 集成测试策略
- C.3 本部分小结与思考

5 D 案例：使用 Google Test 进行单元测试

- D.1 Google Test 测试框架
- D.2 理解 GTest 测试代码编写
- D.3 实例：搭建 GTest 测试环境及测试

6 课后思考及工程实践

小结

1. 软件测试“应以小规模开始，逐步转向大规模”。
2. 设计单元测试主要关注单元本身正确、输入输出正常、数据完整等方面。
3. 执行单元测试需要构建必要的执行环境。
4. 分析单元测试可以使用程序插桩技术。
5. 不同编程语言体系有相应的单元测试工具支持。

思考：单元测试

1. 为什么要进行单元测试？单元测试的主要任务有哪些？
2. 单元测试的对象不可能是一组函数或多个程序的组合，为什么？
3. 单元测试一般由开发人员完成，并采用动态测试技术，这样会获得更高的测试效率和更彻底的测试，谈谈其中的道理。
4. 如何做好单元测试的管理工作？

4

C 集成测试

1 本讲学习目的与内容

2 A 从软件生命周期角度来看测试

- A.1 软件测试过程模型的发展
- A.2 实施软件测试

3 B 单元测试

- B.1 理解单元测试
- B.2 单元测试执行环境
- B.3 程序插桩技术
- B.4 单元测试工具的运用

■ B.5 本部分小结与思考

4 C 集成测试

- C.1 理解集成测试
- C.2 集成测试策略
- C.3 本部分小结与思考

5 D 案例：使用 Google Test 进行单元测试

- D.1 Google Test 测试框架
- D.2 理解 GTest 测试代码编写
- D.3 实例：搭建 GTest 测试环境及测试

6 课后思考及工程实践

1 本讲学习目的与内容

2 A 从软件生命周期角度来看测试

- A.1 软件测试过程模型的发展
- A.2 实施软件测试

3 B 单元测试

- B.1 理解单元测试
- B.2 单元测试执行环境
- B.3 程序插桩技术
- B.4 单元测试工具的运用

■ B.5 本部分小结与思考

4 C 集成测试

■ C.1 理解集成测试

■ C.2 集成测试策略

■ C.3 本部分小结与思考

5 D 案例：使用 Google Test 进行单元测试

■ D.1 Google Test 测试框架

■ D.2 理解 GTest 测试代码编写

■ D.3 实例：搭建 GTest 测试环境及测试

6 课后思考及工程实践

问题的提出

逻辑上，也与传统制造业类似：**汽车组装——“零件都合格，上路却熄火”**

- ▶ 汽车工厂生产的零件：发动机、变速箱、刹车系统、电路模块，每个零件出厂前都通过了严格的单独检测，全部合格。
- ▶ 然而，当工程师把这些零件组装成一辆完整的汽车后，真正上路可能出现问题。

例如：

- ▶ 发动机正常运转，变速箱也能换挡，但刹车踩下去时，发动机没有自动降速。
- ▶ 原因可能是：发动机控制模块和刹车控制模块之间的信号协议不匹配，等等。

这就是集成测试的核心含义——验证各个合格模块组装在一起后，能否正确“对话”和协同工作。

集成测试对象

在现实软件开发与测试方案中，集成是指多个组件的组合

- ▶ 许多组件组合成子系统；
- ▶ 这些子系统又聚合成更大部分，如子系统或系统。

典型的测试对象：已测组件集成后的子系统或系统

- ▶ 组件与组件是否能协同工作；
- ▶ 子系统内组件与组件间的接口；
- ▶ 子系统与外界的接口，等等。

根据不同的测试对象规模，关注不同的侧重点：

- ▶ **组件集成**：数据是否能够在各组件之间正确传递和调用，以及各组件能否正确地协同工作。
- ▶ **系统集成**：关注不同系统之间的相互通信和协同作用。

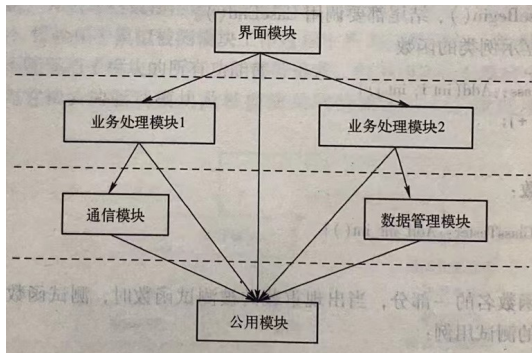
什么是集成测试？

定义：

- 按照系统技术设计的要求将多个已经测试通过的组件组合成一个新的更大的组件（子系统），并检验各部分工作是否达到或实现相应技术指标及要求的**过程**和**活动**。

目标：

- 把经过单元测试的模块拿来，构造一个设计中所描述的场景；
- 旨在暴露接口以及集成组件/系统间交互时存在的缺陷。**



什么是集成测试？

要解决的具体问题：接口连接问题

接口提供方法：API、SDK

- ▶ 函数相互调用
- ▶ 模块间的相互影响
- ▶ 单个模块中的缺陷发生扩散
- ▶ 全局数据结构

- ▶ 内部接口：产品内部两模块之间通信的接口
- ▶ 外部接口：产品之外第三方可以看到的接口
- ▶ 显式接口：写入文档的明确化的接口
- ▶ 隐含接口：未写入文档，只有开发人员知道；如：OS 隐藏的 API

集成测试的具体内容

功能特性：

1. 功能：集成的子系统功能测试，即检查集成的子系统能否满足设计要求的功能特性和指标。
2. 影响：检查一个软件组件的功能是否会对另一个软件组件的功能产生不利影响。
3. 精度：根据计算精度的要求，多个软件组件的误差积累起来，是否仍能够达到设计要求的技术指标。
4. 接口：软件组件之间的接口测试，即集成的子系统内各软件组件之间，数据在通过其接口时是否会出现不一致情况，例如是否会出现数据丢失。
5. 数据：全局数据结构的测试，即检查各个软件组件所用到的全局变量是否一致、合理。

非功能特性：

1. 安全：对程序中可能有的特殊安全性要求进行测试。
2. 其它：根据软件需求和设计中提出的非功能特性的要求进行非功能性测试。

集成测试中采用的测试方法

测试方法

- ▶ 黑盒测试技术为主
- ▶ 白盒测试技术为辅

测试策略

- ▶ 在集成测试过程中，测试人员应该把精力集中在集成本身，其测试任务与集成测试策略相关。



集成测试的依据

依据

- 软件和系统技术设计文档、系统架构、定义和描述接口的文档、 workflows、用例等。

例：某行业关于集成测试（本例中称为部件测试）的技术要求

为保证软件部件测试充分和有效,在进行软件部件测试时,应满足以下技术要求。

- (1) 应对软件部件进行必要的静态测试,并先于动态测试。
- (2) 应逐项测试软件设计文档规定的软件部件的功能、性能等特性。
- (3) 软件部件的每个特性应被至少一个正常的测试用例和一个被认

可的异常测试用例覆盖。

(4) 测试用例的输入应至少包括有效等价类值、无效等价类值和边界数据值。

(5) 应采用合适的集成方法(例如增量法),测试组装新的软件部件。

(6) 应测试软件部件之间、软件部件和硬件之间的所有接口。

(7) 应测试软件单元和软件部件之间的所有调用,达到 100% 的测试覆盖率。

(8) 应测试软件部件的输出数据及其格式。

(9) 应测试运行条件(如数据结构、输入/输出通道容量、内存空间、调用频率等)在边界状态下,进而在人为设定的状态下,软件部件的功能和性能。

(10) 应按设计文档要求,对软件部件的功能、性能进行强度测试。

(11) 对安全性关键的软件部件,应对其进行安全性分析,明确每一个危险状态和导致危险的可能原因,并对此进行针对性的测试。

集成测试可能发现的缺陷

1. 数据内容：组件传送了错误的的数据，或没有传送数据，导致接收数据的组件不能操作或崩溃（组件的功能缺陷、接口格式不兼容、协议缺陷等）。
2. 数据解析：通信正常，但是被调用的组件使用不同的方法来解析接收到的数据（组件的功能缺陷、规格说明矛盾或错误的理解）。
3. 传输时间：数据内容传输正确，但是传输的时间错误或传输的延迟（时序问题），或者传输的时间间隔太短（吞吐量、负荷、容量问题）。

通过静态测试很难发现上述缺陷，更适合通过动态测试来开展集成测试。

集成测试停止的条件

集成测试停止：

- ▶ 意味着完成测试计划、发现并修正了错误，综合平衡预算和开发时间。

示例：以下是针对某行业项目的集成测试停止条件

1. 集成测试用例设计已经通过评审。
2. 按照集成部件计划及增量集成策略完成了整个系统的集成测试。
3. 达到了测试计划中关于集成测试所规定的覆盖率的要求。
4. 集成工作版本满足设计定义的各项功能和性能要求。
5. 在集成测试中发现的错误已经得到修改，各级缺陷修复率达到标准。

集成测试与单元测试的比较

1、测试的对象和侧重点不同

- ▶ 单元测试是针对软件基本单元所做的测试。
- ▶ 集成测试则是以模块和子系统为单位进行的测试，主要测试接口间的关系。

2、测试的依据不同

- ▶ 单元测试是针对软件详细设计所做的测试，测试用例主要依据详细设计。
- ▶ 集成测试是针对高层（概要）设计所做的测试，测试用例主要依据概要设计。

集成测试与单元测试的比较

3、测试空间不同

- ▶ 集成测试不关心内部实现层的测试空间，只关注接口层的测试空间，即关注的是接口层可变数据间的组合关系。
- ▶ 集成测试无法测试从外部输入层的测试空间向接口层测试空间转换时出现的问题，但是可以测试从接口层空间向内部实现层空间进行转换时出现的问题。

4、使用的方法不同

- ▶ 集成测试关注的是接口的集成，单元测试关注单个单元功能测试。
- ▶ 相应地，采用不同的测试用例设计方法。

1 本讲学习目的与内容

2 A 从软件生命周期角度来看测试

- A.1 软件测试过程模型的发展
- A.2 实施软件测试

3 B 单元测试

- B.1 理解单元测试
- B.2 单元测试执行环境
- B.3 程序插桩技术
- B.4 单元测试工具的运用

■ B.5 本部分小结与思考

4 C 集成测试

■ C.1 理解集成测试

■ C.2 集成测试策略

■ C.3 本部分小结与思考

5 D 案例：使用 Google Test 进行单元测试

- D.1 Google Test 测试框架
- D.2 理解 GTest 测试代码编写
- D.3 实例：搭建 GTest 测试环境及测试

6 课后思考及工程实践

关于集成测试策略的认识

通常的认识：集成范围越大，定位缺陷就越困难，从而系统的风险就越大。

基本思路：

- ▶ 集成程度应该逐步增加，而不是一下子将组件或系统集成成为“巨无霸”进行测试。
- ▶ 集成测试策略可以根据系统的框架（例如自顶向下或自底向上）、功能任务集或处理过程顺序等来制定。
- ▶ 如果集成测试计划是在组件或系统生成之前制定的，则可以根据最有效率的测试顺序来进行集成测试。

常用的集成测试策略

制定实用高效的集成测试策略，考虑以下因素：

- ▶ 采用适度的模块耦合度，降低集成复杂性
- ▶ 逐步增加集成的组件数量，容易定位和修正错误

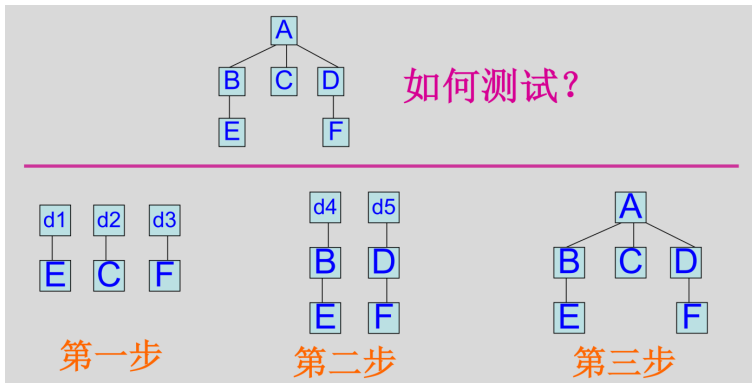
在实际运用中，需要根据被测对象的逻辑层次特性和物理分布特性综合考虑。

常用的具体策略包括：

1. 自底向上集成测试
2. 自顶向下集成测试
3. 核心系统先行集成测试
4. 混合式集成测试
5. 端到端集成测试
6. 随意集成测试（按组件完成时间）
7. 大爆炸集成测试

策略 1: 自底向上集成测试策略 (Bottom-Up Integration)

核心思想: 大多数情况下, 最底层的软件模块会最先完成, 自底向上集成测试是自然的一个选择。



策略 1：自底向上集成测试策略

子组件：对于一个给定层次的组件，它的下层的组件称为子组件，子组件（包括子组件的所有下属组件）事先已经完成集成并经过测试。

策略：

- ▶ 从组件结构中最底层的组件开始集成和测试
- ▶ 最底层组件 -> 子组件 -> ……-> 软件系统

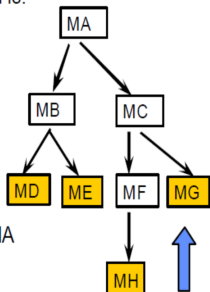
集成步骤：

1. 按照系统技术设计，明确有哪些被测组件。在熟悉被测组件特征的基础上对被测组件进行分层，在同一层次上的组件，可以并行进行测试，然后排出测试活动的先后关系，制定测试执行进度。
2. 按组件间的层次关系，将软件组件集成为子系统，并测试在集成过程中出现的问题。这里，可能需要测试人员开发一些驱动模块或程序来驱动集成活动中形成的被测组件
3. 持续将上层软件组件集成到子系统中，直到集成为完整的系统。

策略 1：自底向上集成测试策略

A possible sequence of integration is:

- Unit test MD
- Unit test ME
- Unit test MH
- Unit test MG
- Integration test MD, MB
- Integration test MD, MB, ME
- Integration test MD, MB, ME, MA
- Integration test MH, MF
- Integration test MH, MF, MC
- Integration test MH, MF, MC, MG
- Integration test all modules



优点：

- ▶ 无需编写桩模块或程序
- ▶ 优先确认底层行为（一个系统中的底层组件往往是比较敏感或关键的组件）

缺点：

- ▶ 需编写驱动模块或程序
- ▶ 推迟确认高层行为（只能通过驱动模块而不是通过用户界面来操作集成的子系统，所以测试会变得复杂烦琐）

策略 2：自顶向下集成测试策略 (Up-Bottom Integration)

核心思想：

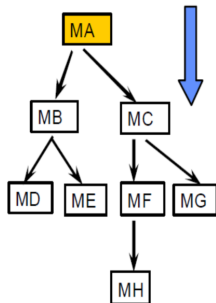
- ▶ 在有些软件开发过程中，往往从最顶层的主控组件（例如，友好方便的图形用户界面 GUI）开始，按照软件的控制层次结构，逐步把各个组件集成在一起。
- ▶ 测试与开发同步。

集成方式：

- ▶ 深度优先：ABDECFHG
- ▶ 广度优先：ABCDEFGH

A possible sequence of integration is:

- Unit test MA
- Integration test MA, MB
- Integration test MA, MB, MC
- Integration test MA, MB, MC, MD
- Integration test MA, MB, MC, MD, ME
- Integration test MA, MB, MC, MD, ME, MF
- Integration test MA, MB, MC, MD, ME, MF, MG
- Integration test all modules



策略 2：自顶向下集成测试策略

测试步骤：

1. 以主控组件作为驱动程序，把对主控组件进行单元测试时引入的桩用实际组件替代。
2. 依据所选的集成策略（深度优先或广度优先），每次只替代一个桩模块。
3. 每集成一个新组件就立即测试一遍。
4. 只有每一次测试完成后，才着手替换下一个桩模块。
5. 为避免引入新错误，必须不断地进行**回归测试**（即全部或部分地重复已做过的测试）。
6. 从步骤 2 开始，循环执行上述步骤，直至整个程序结构集成完毕。

策略 2：自顶向下集成测试策略

优点：

- ▶ 能尽早地对程序主要控制路径和决策机制进行检验，因此能够较早地发现高层控制和决策错误。
- ▶ 不需要专门开发上层的驱动器模块（已测较高级别组件组成了测试环境的主要部分）。
- ▶ 由于是从最上层的组件开始集成，可以直接从（图形）用户界面操作子系统，使测试变得简便。

缺点：

- ▶ 在测试高层组件时，需要模拟低层的桩模块，不能反映真实情况，重要数据不能及时回送到上层组件（测试用例的输入和输出很难明确表示），因此测试并不充分。
- ▶ 桩的成本很高。
- ▶ 底层的敏感和关键组件不能优先集成和测试。

策略 3：核心系统先行集成测试策略

核心思想：

- ▶ 核心组件相对重要且模块耦合度较高，出现问题的可能性较高，尽早测试会相对降低风险。

策略：

- ▶ 先对核心组件进行测试，在测试通过的基础上再按其他组件的重要程度逐个集成到核心系统中；
- ▶ 每次加入一个组件，直至最后形成稳定的软件产品。

对应的集成过程是一个逐渐趋于闭合的螺旋形曲线，代表产品逐步定型的过程。

策略 3：核心系统先行集成测试策略

测试步骤：

1. 对核心系统中的组件进行单独而充分的测试，必要时使用驱动器和桩模块。
2. 将核心系统中的组件一次性集成到被测系统中，解决集成中出现的各类问题。在核心系统规模相对较大的情况下，也可以按照自底向上或自顶向下的步骤，集成核心系统的各组成部分。
3. 按照其他组件的重要程度以及组件间的相互制约关系，拟定其他组件集成到核心系统中的顺序方案。方案经评审以后，即可进行其他组件的集成。
4. 在其他软件组件添加到核心系统以前，首先应该确保它们都已完成组件测试。
5. 按顺序不断加入其他软件组件，修复集成过程中发现的缺陷，形成最终的软件产品。

策略 3：核心系统先行集成测试策略

优点：

- ▶ 对于快速软件开发很有效果，适合较复杂系统的集成测试，能优先保证一些重要功能的实现。

缺点：

- ▶ 采用此策略的系统一般应能明确区分核心软件组件。
- ▶ 核心软件组件应具有较高的耦合度；而其他软件组件内部应具有较高的内聚度，但应具有较低的耦合度。

策略 4：混合式集成测试策略

核心思想：

- ▶ 结合开发和测试团队实际来灵活安排

特点：

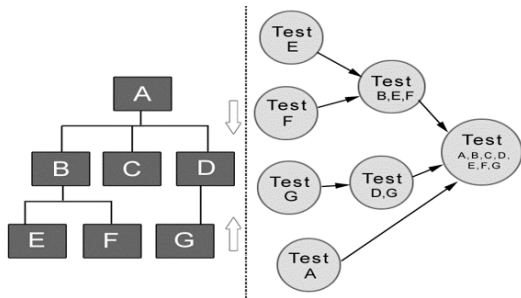
- ▶ 开发小组对各自的低层模块向上集成
- ▶ 专门的集成小组进行自顶向下集成

步骤：

1. 用程序桩独立测试上层模块
2. 用驱动器独立测试低层模块
3. 集成时对中间层进行测试

策略：

- ▶ 综合自顶向下和自底向上，是实际测试中的实用集成测试策略。



策略 5：端到端集成测试策略

核心思想：

- ▶ **系统集成测试**更关注不同系统之间的相互协同作用，各开发团队只控制自己开发的系统功能，可能是跨平台的，变更不稳定。

策略：

- ▶ 从最终用户的角度出发，强调系统的功能测试；
- ▶ 验证一组相互连接的系统形成的集成系统是否正常运行。

适用领域：

- ▶ 大型软件系统



策略 6：随机集成测试（按组件完成时间）

核心思想：

- ▶ 在条件具备时，尽快开展集成测试。

策略：

- ▶ 当组件通过单元测试后，即可开始集成测试。
- ▶ 在此过程中，如果有多个组件同时进入集成测试就绪点，也可采用前述的策略来选择。

集成测试步骤：

- ▶ 按照组件完成的顺序进行集成。

优点：

- ▶ 节省时间，因为每个组件可以最快地集成到系统中。

缺点：

- ▶ 需要设计桩和测试驱动器，代价较大。

策略 7：大爆炸集成测试策略

核心思想：

- ▶ 强调全面快速集成。

问题：

- ▶ 无休止的错误：如错误很多，错误修复很困难；修正错误后，新的错误马上出现。
- ▶ 模块一次性结合，难以找出错误原因。
- ▶ 接口错误和其它错误容易混淆。

适用领域：小型软件开发。

策略： 又称**瞬时集成测试**

- ▶ 当所有构件都通过单元测试，就把它组合成一个最终系统，并观察它是否正常运转。



集成测试策略的灵活运用

在实际操作中，一个项目可能同时采用多种测试策略：

- ▶ 例如，主要采用基于核心系统先行集成策略，但是在其中的一些子系统中采用自底向上的策略。

应该结合项目的实际工程环境和各测试策略适用的范围进行合理的集成测试策略选型。

考虑的因素	方法建议
设计和需求清晰	自顶向下
需求、设计和体系结构不断变化	自底向上
体系结构改变、设计稳定	混合式
体系结构较小，规模小	大爆炸集成
体系结构较复杂，快速开发	核心系统先行集成、随机集成
体系结构复杂，软件规模大	混合式、端到端集成

1 本讲学习目的与内容

2 A 从软件生命周期角度来看测试

- A.1 软件测试过程模型的发展
- A.2 实施软件测试

3 B 单元测试

- B.1 理解单元测试
- B.2 单元测试执行环境
- B.3 程序插桩技术
- B.4 单元测试工具的运用

■ B.5 本部分小结与思考

4 C 集成测试

- C.1 理解集成测试
- C.2 集成测试策略

■ C.3 本部分小结与思考

5 D 案例：使用 Google Test 进行单元测试

- D.1 Google Test 测试框架
- D.2 理解 GTest 测试代码编写
- D.3 实例：搭建 GTest 测试环境及测试

6 课后思考及工程实践

小结

- ▶ 集成测试是按照系统技术设计的要求将多个已经测试通过的组件组合成一个新的更大的组件（子系统）的过程。
- ▶ 集成测试与单元测试各负其责，测试单元、依据、测试空间和使用的测试方法不同。
- ▶ 根据被测对象的逻辑层次特性和物理分布特性综合考虑，灵活选择集成测试策略。

思考：集成测试

1. 集成测试策略多样化，不同的策略适应不同的情况。在实际测试项目中，选择集成测试策略应考虑哪些因素？
2. 集成测试策略一经确定，就不应再做根本性调整，这样的认识是否存在问题？
3. 如果要尽可能敏捷、快速地开展集成测试，应该制定什么样的集成测试策略？
4. 关于集成测试的论点，下面哪些描述是错误的？
A. 集成测试的范围越大，从特定组件或系统中分离出失效就越困难。
B. 集成测试只有自顶向下或自底向上两种集成方法。
C. 为了减少在生命周期后期发现缺陷而产生的风险，集成程度应该逐步增加。
D. 集成测试时，测试人员应该重点关注组件之间的接口。

5

D 案例：使用 Google Test 进行单元测试

1 本讲学习目的与内容

2 A 从软件生命周期角度来看测试

- A.1 软件测试过程模型的发展
- A.2 实施软件测试

3 B 单元测试

- B.1 理解单元测试
- B.2 单元测试执行环境
- B.3 程序插桩技术
- B.4 单元测试工具的运用

■ B.5 本部分小结与思考

4 C 集成测试

- C.1 理解集成测试
- C.2 集成测试策略
- C.3 本部分小结与思考

5 D 案例：使用 Google Test 进行单元测试

- D.1 Google Test 测试框架
- D.2 理解 GTest 测试代码编写
- D.3 实例：搭建 GTest 测试环境及测试

6 课后思考及工程实践

1 本讲学习目的与内容

2 A 从软件生命周期角度来看测试

- A.1 软件测试过程模型的发展
- A.2 实施软件测试

3 B 单元测试

- B.1 理解单元测试
- B.2 单元测试执行环境
- B.3 程序插桩技术
- B.4 单元测试工具的运用

■ B.5 本部分小结与思考

4 C 集成测试

- C.1 理解集成测试
- C.2 集成测试策略
- C.3 本部分小结与思考

5 D 案例：使用 Google Test 进行单元测试

- D.1 Google Test 测试框架
- D.2 理解 GTest 测试代码编写
- D.3 实例：搭建 GTest 测试环境及测试

6 课后思考及工程实践

Google Test 简介

GoogleTest

- ▶ Google 开源的一个跨平台 C++ 单元测试框架，可以帮助程序员验证 C++ 程序的结果预期。

GTest 官网：

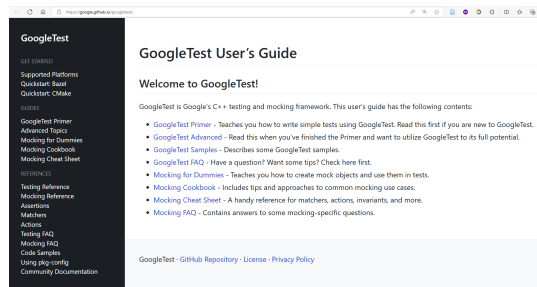
<https://google.github.io/googletest/>

Git 仓库：

<https://github.com/google/googletest>



邵宇 @linux



Google Test 的主要特点

1、测试用例相对独立

- ▶ 每个测试用例运行在不同的对象上，从而使测试之间相对独立
- ▶ 当测试失败时，允许单独运行该对象以进行快速调试

2、高效率执行测试

- ▶ 当测试用例运行失败时，可提供尽可能多的有效信息以便快速定位
- ▶ 不会在第一次测试失败时停止，只会停止当前测试并继续下一个测试

3、提供丰富的测试接口

- ▶ 断言、参数化、致命与非致命判断等

Google Test 的应用

使用 Google Test 的内部项目

- ▶ Google 内部的许多项目
- ▶ Chrome 浏览器和 Chrome OS 背后
- ▶ LLVM 编译器
- ▶ Protocol Buffers, Google 的数据交换格式
- ▶ OpenCV 的计算机视觉库

Google Test 的应用

与 Google Test 相关的开源项目

- ▶ GTest Runner：一个基于 Qt5 的自动化测试运行器
- ▶ GoogleTest UI：一个测试运行器，可测试二进制文件
- ▶ GTest TAP Listener：GoogleTest 的一个事件监听器，实现了测试结果输出的 TAP 协议
- ▶ gtest-parallel：一个测试运行器，能够并行运行二进制文件的测试
- ▶ GoogleTest Adapter：一个 VS Code 扩展
- ▶ C++ TestMate：也是一个 VS Code 扩展
- ▶ Cornichon：小型 Gherkin DSL 解析器，可为 GoogleTest 生成存根代码

1 本讲学习目的与内容

2 A 从软件生命周期角度来看测试

- A.1 软件测试过程模型的发展
- A.2 实施软件测试

3 B 单元测试

- B.1 理解单元测试
- B.2 单元测试执行环境
- B.3 程序插桩技术
- B.4 单元测试工具的运用

■ B.5 本部分小结与思考

4 C 集成测试

- C.1 理解集成测试
- C.2 集成测试策略
- C.3 本部分小结与思考

5 D 案例：使用 Google Test 进行单元测试

- D.1 Google Test 测试框架
- D.2 理解 GTest 测试代码编写
- D.3 实例：搭建 GTest 测试环境及测试

6 课后思考及工程实践

实施单元测试时，我们关注哪些方面的测试代码？

思考：

- ▶ 如果要对软件单元的代码执行测试，在我们编写测试代码时应该涉及到哪些方面？

P1: 1→11

- ▶ 用例 1: $nPosX = -1$, $nPosY = -$

P2: 1→2,3→6→7→9→10→1→11

- ▶ 用例 2: $nPosX = 1$, $nPosY = 1$

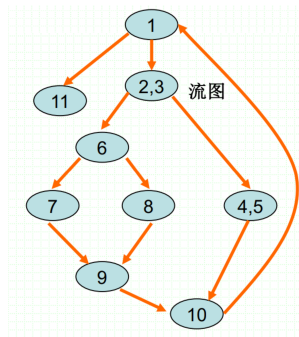
P3: 1→2,3→4, 5→10→1→11

- ▶ 用例 3: $nPosX = 1$, $nPosY = -1$

P4: 1→2,3→6→8→9→10→1→11

- ▶ 用例 4: $nPosX = 1$, $nPosY = -3$

```
void Func(int nPosX, int nPosY) {  
    while (nPosX > 0) {  
        int nSum = nPosX + nPosY;  
        if (nSum > 1) {  
            nPosX--;  
            nPosY--;  
        }  
        else {  
            if (nSum < -1) nPosX -= 2;  
            else nPosX = 4;  
        }  
    }  
    // end of while  
}
```



实施单元测试时，我们关注哪些方面的测试代码？

1. 如何表达测试用例？断言，表达程序某些特性来说明程序正确性（例如输入输出）。

实施单元测试时，我们关注哪些方面的测试代码？

1. **如何表达测试用例？** 断言，表达程序某些特性来说明程序正确性（例如输入输出）。
2. **如何执行测试用例？** 事件，通过事件机制，在不同时机自动触发对测试前或测试后的处理动作。

实施单元测试时，我们关注哪些方面的测试代码？

1. 如何表达测试用例？断言，表达程序某些特性来说明程序正确性（例如输入输出）。
2. 如何执行测试用例？事件，通过事件机制，在不同时机自动触发对测试前或测试后的处理动作。
3. 如何传递测试数据？参数化，给程序传递参数。

1. 编写断言（例 7：加法运算）

断言：在程序特定部位插入用以证实程序某些特性的语句。

源文件：add.cpp

```
#include <iostream>
#include "gtest/gtest.h"

#pragma comment (lib, "gtestd.lib")

int add (int a, int b)
{
    return a + b;
}

// 编写测试用例

TEST (testCase, test0)
{
    EXPECT_EQ ( add(2, 3), 5); //判断结果是不是等于5, EXPECT_EQ表示"等于"
}

int main (int argc, char** argv)
{
    testing::InitGoogleTest(&argc, argv);
    return RUN_ALL_TESTS();
}
```

1. 编写断言（例 8：求开方运算）

```
#include "sqrt.h"

int sqrt(int x){
    if(x<=0) return 0;
    if(x==1) return 1;

    int small=0;
    int large=x;
    int temp=x/2;

    while(small<large) {
        int a = x/temp;
        int b = x/(temp+1);

        if (a==temp) return a;
        if (b==temp+1) return b;

        if (temp<a && temp+1>b) {
            return temp;
        }
        else if (temp<a && temp+1<b) {
            small=temp+1;
            temp = (small+large)/2;
        }
        else {
            large = temp;
            temp = (small+large)/2;
        }
    }
    return -1;
}
```

（本页的代码在课后工程实践中作为例子使用）

← sqrt.cpp
sqrt.h

```
#ifndef _SQRT_H_
#define _SQRT_H_

int sqrt(int x);

#endif // _SQRT_H_
```

sqrt-unittest.cpp

```
#include "sqrt.h"
#include "gtest/gtest.h"
```

```
TEST(SQRTTest, Zero) {
    EXPECT_EQ(0, sqrt(0));
}
```

```
TEST(SQRTTest, Positive) {
    EXPECT_EQ(100, sqrt(10000));
    EXPECT_EQ(1000, sqrt(1000000));
    EXPECT_EQ(99, sqrt(9801));
}
```

```
TEST(SQRTTest, Negative){
    int i=-1;
    EXPECT_EQ(0, sqrt(i));
}
```

1. 编写断言

要测试一个类或函数，需要对其行为做出断言。当一个断言失败时：

- ▶ Google Test 会在屏幕上输出该代码所在的源文件及其所在的位置行号，以及错误信息
- ▶ 开发人员也可以提供一个自定义的错误信息，这个信息在失败时会被附加在 Google Test 的错误信息之后

断言常常成对出现，它们都测试同一个类或者函数，但对当前功能有着不同的效果：

- ▶ ASSERT_* 版本的断言失败时会产生致命失败，并结束当前函数
- ▶ EXPECT_* 版本的断言产生非致命失败，而不会中止当前函数

编写测试代码时：

- ▶ 通常更推荐使用 EXPECT_* 断言，因为它们运行一个测试中可以有不止一个的错误被报告出来。
- ▶ 但是，如果在编写断言失败就没有必要继续往下执行的测试用例时，应该使用 ASSERT_* 断言。

1. 编写断言：GTest 提供的断言类型

1、布尔值检查

Fatal assertion	Nonfatal assertion	Verifies
<code>ASSERT_TRUE(condition);</code>	<code>EXPECT_TRUE(condition);</code>	<i>condition is true</i>
<code>ASSERT_FALSE(condition);</code>	<code>EXPECT_FALSE(condition);</code>	<i>condition is false</i>

2、数值型数据检查

Fatal assertion	Nonfatal assertion	Verifies
<code>ASSERT_EQ(expected, actual);</code>	<code>EXPECT_EQ(expected, actual);</code>	<i>expected == actual</i>
<code>ASSERT_NE(val1, val2);</code>	<code>EXPECT_NE(val1, val2);</code>	<i>val1 != val2</i>
<code>ASSERT_LT(val1, val2);</code>	<code>EXPECT_LT(val1, val2);</code>	<i>val1 < val2</i>
<code>ASSERT_LE(val1, val2);</code>	<code>EXPECT_LE(val1, val2);</code>	<i>val1 <= val2</i>
<code>ASSERT_GT(val1, val2);</code>	<code>EXPECT_GT(val1, val2);</code>	<i>val1 > val2</i>
<code>ASSERT_GE(val1, val2);</code>	<code>EXPECT_GE(val1, val2);</code>	<i>val1 >= val2</i>

3、字符串比较

Fatal assertion	Nonfatal assertion	Verifies
<code>ASSERT_STREQ(expected_str, actual_str);</code>	<code>EXPECT_STREQ(expected_str, actual_str);</code>	两个C字符串有相同的内容
<code>ASSERT_STRNE(str1, str2);</code>	<code>EXPECT_STRNE(str1, str2);</code>	两个C字符串有不同的内容
<code>ASSERT_STRCASEEQ(expected_str, actual_str);</code>	<code>EXPECT_STRCASEEQ(expected_str, actual_str);</code>	两个C字符串有相同的内容，忽略大小写
<code>ASSERT_STRCASENE(str1, str2);</code>	<code>EXPECT_STRCASENE(str1, str2);</code>	两个C字符串有不同的内容，忽略大小写

1. 编写断言：GTest 提供的断言类型

4、异常检查

Fatal assertion	Nonfatal assertion	Verifies
<code>ASSERT_THROW(statement, exception_type);</code>	<code>EXPECT_THROW(statement, exception_type);</code>	<i>statement</i> throws an exception of the given type
<code>ASSERT_ANY_THROW(statement);</code>	<code>EXPECT_ANY_THROW(statement);</code>	<i>statement</i> throws an exception of any type
<code>ASSERT_NO_THROW(statement);</code>	<code>EXPECT_NO_THROW(statement);</code>	<i>statement</i> doesn't throw any exception

5、浮点型检查

Fatal assertion	Nonfatal assertion	Verifies
<code>ASSERT_FLOAT_EQ(expected, actual);</code>	<code>EXPECT_FLOAT_EQ(expected, actual);</code>	the two float values are almost equal
<code>ASSERT_DOUBLE_EQ(expected, actual);</code>	<code>EXPECT_DOUBLE_EQ(expected, actual);</code>	the two double values are almost equal

对相近的两个数比较：

Fatal assertion	Nonfatal assertion	Verifies
<code>ASSERT_NEAR(val1, val2, abs_error);</code>	<code>EXPECT_NEAR*(val1, val2, abs_error*);</code>	the difference between <i>val1</i> and <i>val2</i> doesn't exceed the given absolute error

6、此外还有类型检查、谓词检查等

2. 定义事件机制（例 9：单元测试代码结构）

通过定义事件，可以在执行测试用例之前或之后执行设定的操作，以获得想要的信息。

```
[=====] Running 1 test from 1 test suite.  
[-----] Global test environment set-up.  
Foo FooEnvironment SetUp  
[-----] 1 test from TestMap  
SetUpTestCase()  
[ RUN      ] TestMap.Test1  
SetUp() is running  
***** 23 *****  
TearDown()  
[       OK ] TestMap.Test1 (0 ms)  
TearDownTestCase()  
[-----] 1 test from TestMap (0 ms total)  
  
[-----] Global test environment tear-down  
Foo FooEnvironment TearDown  
[=====] 1 test from 1 test suite ran. (3 ms total)  
[  PASSED  ] 1 test.
```

2. 定义事件机制（例 9：单元测试代码结构）

```
#include <iostream>
#include <map>
#include "gtest/gtest.h"

#pragma comment(lib, "gtestd.lib")
using namespace std;

class Student {
public:
    Student() {
        age = 0;
    }
    Student(int a) {
        age = a;
    }
    void print() {
        cout << "***** " << age << " *****" << endl;;
    }
private:
    int age;
};

class FooEnvironment: public testing::Environment {
public:
    virtual void SetUp() {
        std::cout << "Foo FooEnvironment SetUp" << std::endl;
    }
    virtual void TearDown() {
        std::cout << "Foo FooEnvironment TearDown" << std::endl;
    }
};

static Student* s;
```

```
/* 在第一个test之前，最后一个test之后调用
   SetUpTestCase()和TearDownTestCase() */
class TestMap: public testing::Test {
public:
    static void SetUpTestCase() {
        cout << "SetUpTestCase() " << endl;
        s = new Student(23);
    }
    static void TearDownTestCase() {
        delete s;
        cout << "TearDownTestCase()" << endl;
    }
    void SetUp() {
        cout << "SetUp() is running" << endl;
    }
    void TearDown() {
        cout << "TearDown() " << endl;
    }
};

TEST_F(TestMap, Test1) {
    // you can refer to s here
    s->print();
}

int main(int argc, char** argv) {
    testing::AddGlobalTestEnvironment(new FooEnvironment);
    testing::InitGoogleTest(&argc, argv);
    return RUN_ALL_TESTS();
}
```

2. 定义事件机制

GTest 提供三类事件：

- ▶ 全局事件：所有测试用例执行之前或之后，产生该事件；
- ▶ TestSuite 事件（测试套件）：在某一个测试套件（一批测试用例）中第一个测试用例之前、最后一个测试用例之后，产生该事件；
- ▶ TestCase 事件（测试用例）：在某一个测试用例之前或之后，产生该事件。

通过继承 Test 类，使用 TEST_F 宏，可以在用例之间共享一些通用的方法。

2. 事件机制：编写全局事件

(1) 要实现全局事件，需要写一个类，继承 `testing::Environment` 类，实现里面的 `SetUp` 和 `TearDown` 方法：

- ▶ `SetUp()`：在所有测试用例执行前执行
- ▶ `TearDown()`：在所有测试用例执行后执行

(2) 在 GTest 框架中添加这个全局事件：

- ▶ 在 `main` 函数中通过 `testing::AddGlobalTestEnvironment` 方法将事件挂进来
- ▶ 如果编写很多个这样的类，可以将其事件都挂上去

2. 事件机制：编写 TestSuite 事件（测试套件）

- (1) 要实现 TestSuite 事件，需要写一个类，继承 `testing::Test`，实现两个静态方法：
- ▶ `SetUpTestCase()`：在第一个 `TestCase` 之前执行
 - ▶ `TearDownTestCase()`：在最后一个 `TestCase` 之后执行
- (2) 在编写测试套件时，需要使用 `TEST_F` 这个宏，第一个参数是上面类的名字，代表一个测试套件（TestSuite）。

2. 事件机制：编写 TestCase 事件（测试用例）

TestCase 事件是挂在每个测试用例执行前后的，实现方式类似，需要实现 SetUp 方法和 TearDown 方法：

- ▶ SetUp(): 在每个 TestCase 之前执行
- ▶ TearDown(): 在每个 TestCase 之后执行

3. 参数化实现（例 10：被测函数参数传递）

如果需要多次为被测函数传入不同的值的情况，这时可以通过参数化来进行测试。

```
#include <gtest/gtest.h>

// Returns true if n is a prime number.
bool IsPrime(int n) {
    // Trivial case 1: small numbers
    if (n <= 1) return false;
    // Trivial case 2: even numbers
    if (n % 2 == 0) return n == 2;
    // Now, we have that n is odd and n >= 3.
    // Try to divide n by every odd number i, starting from 3
    for (int i = 3; i <= sqrt(n); i += 2) {
        // We only have to try i up to the square root of n
        if (n % i == 0) break;
        // Now, we have i <= n/i < n.
        // If n is divisible by i, n is not prime.
        if (n % i == 0) return false;
    }
    // n has no integer factor in the range (1, n), and thus is prime.
    return true;
}
```

判断是否素数的函数

```
class IsPrimeParamTest: public::testing::TestWithParam<int>{};

TEST_P(IsPrimeParamTest, HandleTrueReturn) {
    int n = GetParam();
    EXPECT_TRUE(IsPrime(n));
}

// 被测函数须传入多个相关的值
INSTANTIATE_TEST_CASE_P(TrueReturn, IsPrimeParamTest,
    testing::Values(3, 5, 11, 23, 17));

int main (int argc, char **argv) {
    testing::InitGoogleTest (&argc, argv);
    return RUN_ALL_TESTS();
}
```

3. 参数化实现

具体方法：

- ▶ 添加一个类，继承 `testing::TestWithParam`
- ▶ 加入需要参数化的参数类型（如前述案例中的 `int` 型参数）

4. 使用 GCOV/LCOV 进行代码覆盖率统计

GCOV/LCOV 是与 GCC 配套的代码覆盖率统计工具。

用法如下：

1. 通过 gcc 加上编译参数："`fprofile-arcs -ftest-coverage -lgcov`", 编译后生成 `gcno` 文件。
2. 可执行文件执行后生成 `gcda` 文件。
3. 执行以下命令，在 `result` 目录下生成 `index.html`：

```
lcov -d . -c -o test.info -b .  
genhtml -o result test.info
```
4. 使用浏览器打开 `index.html` 可看到可视化的代码覆盖率统计结果。

1 本讲学习目的与内容

2 A 从软件生命周期角度来看测试

- A.1 软件测试过程模型的发展
- A.2 实施软件测试

3 B 单元测试

- B.1 理解单元测试
- B.2 单元测试执行环境
- B.3 程序插桩技术
- B.4 单元测试工具的运用

■ B.5 本部分小结与思考

4 C 集成测试

- C.1 理解集成测试
- C.2 集成测试策略
- C.3 本部分小结与思考

5 D 案例：使用 Google Test 进行单元测试

- D.1 Google Test 测试框架
 - D.2 理解 GTest 测试代码编写
 - D.3 实例：搭建 GTest 测试环境及测试
- ## 6 课后思考及工程实践

搭建 GTest 测试环境及测试

作为本讲的课后工程实践，具体操作步骤：

- ▶ 请参考：《Linux 系统下 Google Test 单元测试环境搭建》 (<https://blog.csdn.net/lk07828/article/details/49799263>)
- ▶ 也可参考"GoogleTest: C++ unit test framework"
<http://www.yolinux.com/TUTORIALS/Cpp-GoogleTest.html>

涉及到的示例代码

说明：

- ▶ 源代码 1 中的文件与《Linux 系统下 Google Test 单元测试环境搭建》中的示例代码相同。

源代码 1：求平方根函数

- `sqrt.cpp`
- `sqrt.h`
- `sqrt-unittest.cpp`

6

课后思考及工程实践

1 本讲学习目的与内容

2 A 从软件生命周期角度来看测试

- A.1 软件测试过程模型的发展
- A.2 实施软件测试

3 B 单元测试

- B.1 理解单元测试
- B.2 单元测试执行环境
- B.3 程序插桩技术
- B.4 单元测试工具的运用

■ B.5 本部分小结与思考

4 C 集成测试

- C.1 理解集成测试
- C.2 集成测试策略
- C.3 本部分小结与思考

5 D 案例：使用 Google Test 进行单元测试

- D.1 Google Test 测试框架
- D.2 理解 GTest 测试代码编写
- D.3 实例：搭建 GTest 测试环境及测试

6 课后思考及工程实践

思考

主题：

1. 在测试智能软件时，本讲涉及的单元测试、集成测试技术要求是否还有效、表现在哪些方面？
2. 软件工厂是一种以工业化思维重构软件生产的开发体系，通过标准化、自动化、模块化，能够积累可重复利用的通用架构、组件、模型、工具和功能模板等核心资产库。请说明：软件工厂哪些方面或环节体现了单元测试、集成测试的技术要求？

【提交方式】

- ▶ 通过邮件发送到: rjw@iscas.ac.cn
- ▶ 国庆假期后周五（10.9）晚上 12 点前提交。（与下一讲合并为一个 PPT 文件）

工程实践：搭建 Google Test 单元测试环境并完成代码测试

步骤：

1. 在自己机器的测试实验虚拟机平台中，搭建 Google Test 单元测试环境。
2. 针对开方运算 (sqrt.cpp/sqrt.h/sqrt-unittest.cpp) 实现代码，学习 GTest 单元测试框架和测试代码编写方法。

注意：

- ▶ 参考《Linux 系统下 Google Test 单元测试环境搭建》，按其中的 5 步来操作即可。

参考资料

资料名称	类型	如何获取？
《Linux 系统下 Google Test 单元测试环境搭建》 sqrt.cpp/sqrt.h/sqrt-unittest.cpp	技术文档 源代码	国科大在线班级资料目录 国科大在线班级资料目录

相关链接

了解课程：[课程门户](#)



群聊：软件测试技术与实践课程
群（2026）



该二维码7天内(9月19日前)有效，重新进入将更新

加入微信群

获取课程资料：

1、国科大在线移动端



2、浏览器访问[国科大在线](#)

欢迎指正和建议，谢谢大家！

